Kapitel 4: Algorithmische Grundstrukturen und ihre Darstellung

Bisher haben wir hauptsächlich Programme gesehen, bei denen der Programmcode Zeile für Zeile nacheinander ausgewertet wird. Mit Hilfe **algorithmischer Grundstrukturen** können wir diesem Verhalten abweichen.

Algorithmische Grundstrukturen steuern dabei den Ablauf eines Programms. Mit ihnen könnt ihr Anweisungsblöcke definieren, die nur unter einer bestimmten Bedingung ausgeführt werden oder auch solche, deren Ausführung mehrfach erfolgt. Hierfür unterscheiden wir **Sequenzen**, **Verzweigungen** und **Schleifen**, welche wir uns - inklusiv dreier **Darstellungsformen** - nachfolgend genauer anschauen.

4.8.3. Fußgesteuerte Schleife

Bei der **fußgesteuerten Schleife** wird eine Sequenz so lange wiederholt ausgeführt, **bis eine Bedingung (am Ende der Schleife) erfüllt ist**.

Im **Pseudocode** könnte man exemplarisch folgendes formulieren:

```
WIEDERHOLE
    zaehler = zaehler + 1
BIS zaehler >= 5
```

Am Ende eines jeden Schleifendurchlaufs wird nun überprüft, ob die Schleifenbedingung gültig ist.

Ist dies **nicht der Fall**, wird die Schleife erneut durchlaufen. **Ist die Bedingung jedoch gültig, wird die Wiederholung abgebrochen.**

Da die Bedingung erst am Ende jedes Schleifendurchlaufs überprüft wird, wird so eine Schleife **immmer mindestens einmal ausgeführt**, selbst wenn die Bedingung bereits zu Beginn erfüllt war.

An sich gibt es diese Struktur in Python nicht. Allerdings können wir sie uns konstruieren:
Mit Hilfe einer **Verzweigung** überprüfen wir, ob die Schleifenbedingung gültig ist. Wenn wir mit der Verzweigung feststellen, dass die Schleifenbedingung endlich gültig ist, können wir mit dem Schlüsselwort `break` die wiederholte Ausführung der Schleife verhindern und diese abbrechen.

Damit die Schleife allerdings überhaupt wiederholt ausgeführt werden kann, ist die eigentliche Bedingung hinter dem `WHILE`-Schlüsselwort `True`.

Programmcode:

```python
zaehler = 0

while True:
    zaehler = zaehler + 1
    if zaehler >= 5:
        break
```

Strutkogramm:



Die Darstellung ist sehr ähnlich zur kopfgesteuerten Schleife, allerdings steht die Bedingung nun unten und ist invertiert.

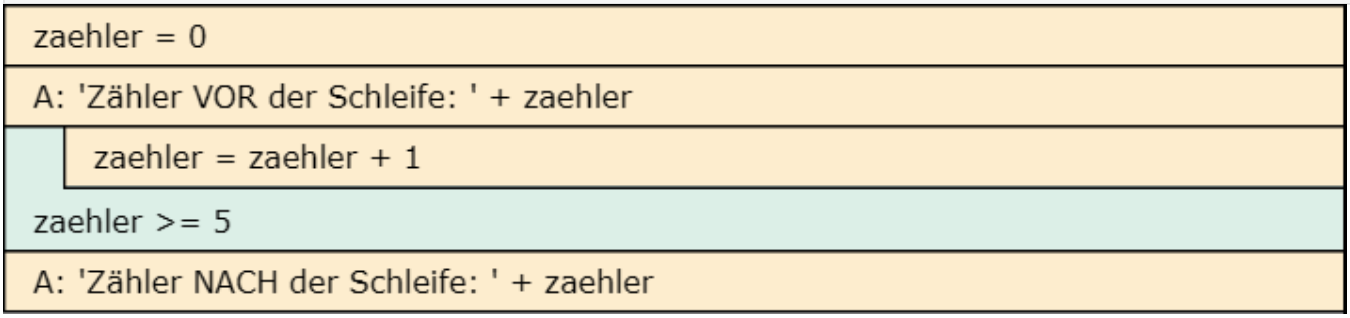
Programmcode und Struktogramm mit Ausgaben zur besseren Nachvollziehbarkeit:

```
# FÜHRE MICH UNBEDINGT AUS!

zaehler = 0
print(f'Zähler VOR der Schleife: {zaehler}')

while True:
    zaehler = zaehler + 1
    if zaehler >= 5:
        break

print(f'Zähler NACH der Schleife: {zaehler}')
```



Programmcode und Struktogramm mit erweiterten Ausgaben zur besseren Nachvollziehbarkeit:

```
# FÜHRE MICH UNBEDINGT AUS!

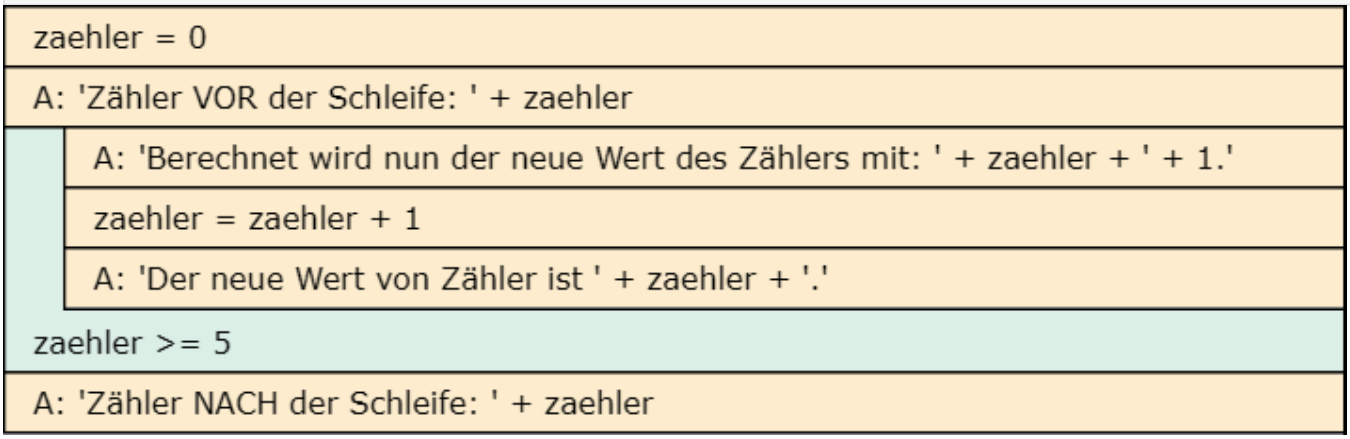
zaehler = 0
print(f'Zähler VOR der Schleife: {zaehler}')

print('- - -')

while True:
    print(f'Berechnet wird nun der neue Wert des Zählers mit: {zaehler} + 1.')
    zaehler = zaehler + 1
    print(f'Der neue Wert von Zähler ist {zaehler}.')
    if zaehler >= 5:
            break

print('- - -')

print(f'Zähler NACH der Schleife: {zaehler}')
```



Hinweise zum Code einer fußgesteuerten Schleife:

- Jede fußgesteuerte Schleife beginnt mit dem Schlüsselwort `while`.
- Statt einer Bedingung folgt nach `while` das Schlüsselwort `True`. Dies entspricht in `PYTHON` dem Wahrheitswert (Boolean) wahr, d. h. statt eine Bedingung zu überprüfen, wird hier sofort das Ergebnis einer solchen Bedingung angegeben: `True`. Theoretisch würde diese Schleife also nie abbrechen.
- Der Schleifenkörper funktioniert analog zu einer normalen kopfgesteuerten Schleife: alle innerhalb eines Schleifendurchlaufs auszuführenden Anweisungen müssen gleich weit eingerückt werden.
- Die Überprüfung der Endbedingung erfolgt mittels einer Verzweigung. Ist die angegebene Bedingung erfüllt, wird die Anweisung break ausgeführt. Dies bewirkt einen Abbruch der gesamten Schleife. Nach dem Abbruch wird wie gewohnt die erste nicht eingerückte Anweisung ausgeführt.

Beispiel 1:

Schauen wir uns das Ganze zum besseren Verständnis nochmal etwas genauer an.

Wir betrachten dazu den folgenden Code.

```python
# FÜHRE MICH UNBEDINGT AUS!

i = 0

while True:
    i = i + 1
    print(i)
    if i >= 10:
        break
```



Was passiert hier eigentlich? Das zeigt für (fast) jeden Schritt die nachfolgende Tabelle.

Schritt	Abbruchbedingung i >= 10?	Aktion	neuer Wert von i
0️⃣	-	Initialisierung (vor der eigentlichen Schleife)	i = 0
1️⃣	❌ FALSCH (0 >= 10)	i = 0 + 1 ➡️ 1	i = 1
2️⃣	❌ FALSCH (1 >= 10)	i = 1 + 1 ➡️ 2	i = 2
3️⃣	❌ FALSCH (2 >= 10)	i = 2 + 1 ➡️ 3	i = 3
...	...	...	...
9️⃣	❌ FALSCH (8 >= 10)	i = 8 + 1 ➡️ 9	i = 9
🔟	❌ FALSCH (9 >= 10)	i = 9 + 1 ➡️ 10	i = 10
⏹️	✅ WAHR (10 >= 10)	Schleife wird verlassen	i = 10 (Endwert)

Die Bedingung `i >= 10` wird am Ende jedes Durchlaufs geprüft. Bei `i >= 10` ist `10 >= 10` wahr und die Schleife bricht ab.

Beispiel 2:

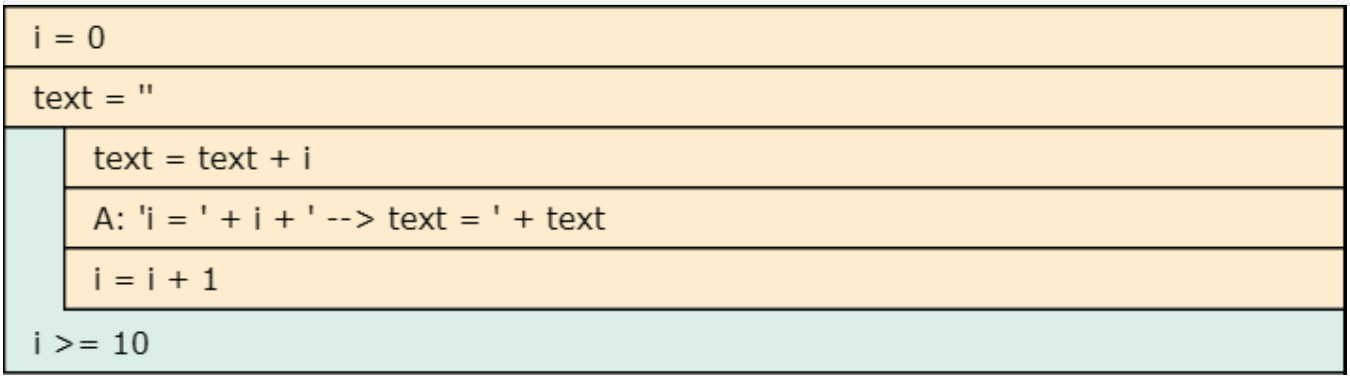
Ein ähnliches Beispiel kann man mit Zeichenketten konstruieren.

Wir behalten unser `i` bei und lassen die Zeichenkette in jedem Schleifendurchlauf wachsen, indem die Zahl als Zeichen hizugefügt wird.

```
# FÜHRE MICH UNBEDINGT AUS!

i = 0
text = ''

while True:
    text = text + str(i)
    print(f'i = {i}   -->   text = {text}')
    i = i + 1
    if i >= 10:
        break
```





4.8.4. Übungen zur fußgesteuerten Schleife

Aufgabe 1

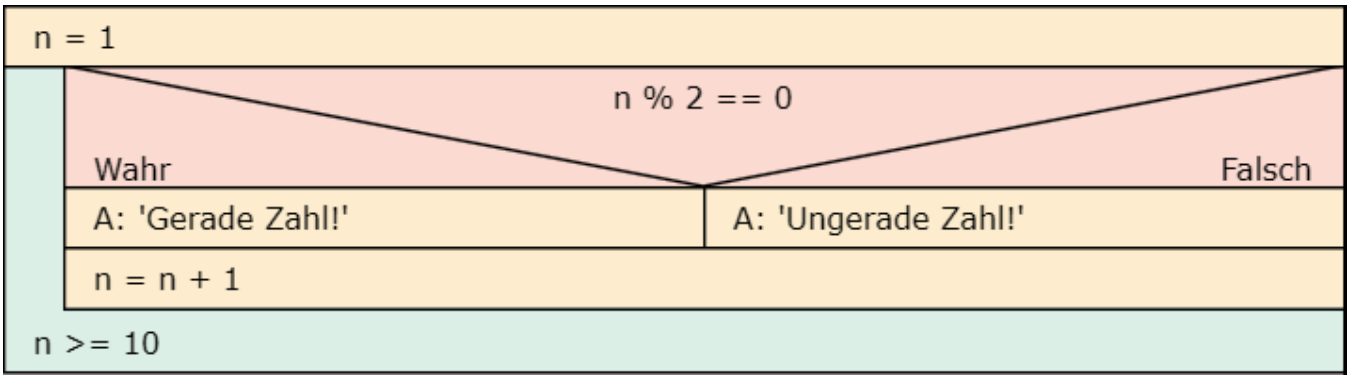
Implementiere den folgenden Pseudocode.

```
Eingabe einer Zahl n
Setze die Variable Zähler auf 1
WIEDERHOLE
    Erhöhe Zähler um 0.125
BIS der Zähler größer n ist
Ausgabe des Zählers
```

```
# HIER IST PLATZ FÜR DEINE LÖSUNG! :)
```

Aufgabe 2

Implementiere das folgende Struktogramm.



Besonders zu beachten ist, dass die letzte Anweisung `n = n + 1` über die gesamte Breite geht. Das bedeutet, dass diese Anweisung **NICHT MEHR** zur Verzweigung gehört!

```
# HIER IST PLATZ FÜR DEINE LÖSUNG! :)
```

Aufgabe 3

Implementiere ein Programm, welches nach Eingabe einer oberen Grenze n, alle Zahlen von 0 bis n aufaddiert.

Bei der Eingabe der Zahl 3 ergibt sich beispielsweise das Ergebnis 6.

Gerechnet wird dabei: $0 + 1 + 2 + 3 = 6$.

Implementiere deine Lösung **zwingend** mit Hilfe einer **fußgesteuerten Schleife**!

```
# HIER IST PLATZ FÜR DEINE LÖSUNG! :)
```

Aufgabe 4

Implementiere ein Programm, welches den Benutzer auffordert, eine Zahl zu erraten.

Die **fußgesteuerte Schleife** soll so lange fortgesetzt werden, bis der Benutzer die richtige Zahl erraten hat.

Gib dem Benutzer nach jeder Eingabe eine Rückmeldung, ob die Zahl zu hoch oder zu niedrig ist.

Hinweis: Verwende eine feste Zahl im Code, die der Benutzer erraten muss.

```
# HIER IST PLATZ FÜR DEINE LÖSUNG! :)
```

Aufgabe 5

Implementiere ein Programm, welches die Zahlwörter eins, zwei, ... , sechs so oft ausgibt, wie es das Wort aussagt, also eins einmal, zwei zweimal.

Implementiere deine Lösung mit Hilfe einer **fußgesteuerten Schleife**! Ein **korrekt funktionierendes Programm** erzeugt die folgende Ausgabe:

```
eins
zwei zwei
drei drei drei
vier vier vier vier
fünf fünf fünf fünf fünf
sechs sechs sechs sechs sechs sechs
```

```
# HIER IST PLATZ FÜR DEINE LÖSUNG! :)
```

© Patrick Binkert & Dr. Stephan Matos Camacho | SJ 26 / 27

